

λ Calculus

杨辰

2023 年 4 月 6 日

1 Origin

1930 Alonzo Church

用来表达计算，能力等价于图灵机。

λ 演算是函数式编程语言的基础。

2 Definition

语法 (syntax):

构成合法程序的写法规则。

语义 (semantics):

描述所写程序的运行行为。

3 Syntax

1. λ 项或 λ 表达式的 BNF 范式:

$M, N ::= x \mid (\lambda x. M) \mid MN$

x 是变量。

2. $(\lambda x. M)$: 即为 lambda 抽象。

比如: $(\lambda x. x + 1)$

这是一个匿名函数 (anonymous), 无法通过函数名进行调用。

3. (MN) : 即为 lambda 作用。

比如: $((\lambda x. x + 1)3)$ 。

注：一些约定：

1. 一般最外层的括号是省略的。

2. λ 主题部分是向后延申的。

比如 $\lambda x MN = \lambda x (MN)$

3. λ 作用是左结合的。

比如 $MNP = (MN)P$

4. 自由变量和绑定变量

比如在 $\lambda x. x + y$ 中， x 是绑定变量， y 是自由变量。

形式化定义：令 $f_v(M)$ 为 M 中的自由变量的集合。则

$$f_v(x) = \{x\}$$

$$f_v(\lambda x. M) = f_v(M) \setminus \{x\}$$

$$f_v(MN) = f_v(M) \cup f_v(N)$$

4 Semantics

1. β 规约

基本规则： $(\lambda x. M)N = M[N/x]$ ，即用 N 来代替 x

2. 替换

$$x[N/x] = N$$

$$y[N/x] = y$$

$$(MP)[N/x] = (M[N/x])(P[N/x])$$

$$(\lambda x. M)[N/x] = \lambda x. M$$

3. 范式

β 可约项 (β -redex): 以 $(\lambda x. M)N$ 出现的 λ 项

β 范式: 没有 β -redex 的 λ 项

4. 汇聚

Church-Rosser theorem

λ 项可以以任意顺序进行规约，只要规约能得到 β 范式结果，结果都是一样的

corollary: 在 α 等价的基础上，任何 λ 表达式最多只有一个 β 范式

5. 规约策略

应用序：总是先规约最左、最内的可规约项

(一个最外的可约项时一个不被其他可约项包含的可约项)

正则序：总是先规约最左、最外的可规约项

theorem: 如果 λ 表达式存在 β 范式，那么按正则序规约总能规约到该 β 范式

5 Church encoding

1. 对 bool 值:

$$\text{True} \stackrel{def}{=} \lambda x. \lambda y. x$$

$$\text{False} \stackrel{def}{=} \lambda x. \lambda y. y$$

2. 相应的操作

$$\text{if } b \text{ then } M \text{ else } N \stackrel{def}{=} b \ M \ N$$

$$3. \text{Not} \stackrel{def}{=} \lambda b. b \ \text{False} \ \text{True}$$

$$\text{Not}' \stackrel{def}{=} \lambda b. \lambda x. \lambda y. b \ y \ x$$

$$4. \text{And} \stackrel{def}{=} \lambda b. \lambda b'. b \ b' \ \text{False}$$

$$\text{Or} \stackrel{def}{=} \lambda b. \lambda b'. b \ \text{True} \ b'$$

5. Church number

$$0 \stackrel{def}{=} \lambda f. \lambda x. x$$

$$1 \stackrel{def}{=} \lambda f. \lambda x. f x$$

$$2 \stackrel{def}{=} \lambda f. \lambda x. f(f x)$$

...

$$n \stackrel{def}{=} \lambda f. \lambda x. f^n x$$

6. 自增

$$\text{Inc} \stackrel{def}{=} \lambda n. \lambda f. \lambda x. f(n \ f \ x)$$

$$\text{Inc} \stackrel{def}{=} \lambda n. \lambda f. \lambda x. n \ f(f \ x)$$

7. 加法

$$\text{Add} \stackrel{def}{=} \lambda n. \lambda m. \lambda f. \lambda x. n \ f(m \ f \ x)$$

8. 乘法

$\text{Multi} \stackrel{\text{def}}{=} \lambda n. \lambda m. \lambda f. \lambda x. n(m\ f)x$

9. 幂运算

$\text{Pow} \stackrel{\text{def}}{=} \lambda n. \lambda n. m(n)$

10. 前驱

$\text{Pred} \stackrel{\text{def}}{=} \lambda n. \lambda f. \lambda x. n(\lambda g. \lambda h. h(g\ f))(\lambda u. x)(\lambda u. u)$

11. 减法

$\text{Sub} \stackrel{\text{def}}{=} \lambda m. \lambda n. n\ \text{Pred}\ m$

12. 是否为 0

$\text{Iszero} \stackrel{\text{def}}{=} \lambda n. n(\lambda x. \text{False})\text{True}$

13. 小于等于

$\text{Leq} \stackrel{\text{def}}{=} \lambda n. \lambda m. n(\lambda x. m(\lambda y. \text{False})\text{True})(\lambda y. \text{True})0$

14. 有序对的编码 (Pairs)

$\text{Pair} \stackrel{\text{def}}{=} \lambda x. \lambda y. \lambda f. f\ x\ y$

$\pi_0 \stackrel{\text{def}}{=} \lambda p. p\text{True}$

$\pi_1 \stackrel{\text{def}}{=} \lambda p. p\text{False}$

15. 对元组的编码 (Tuples)

$\text{Tuple} \stackrel{\text{def}}{=} \lambda x_1. \lambda x_2. \dots \lambda x_n. \lambda f. f(M_1 M_2 \dots M_n)$

$\pi_i \stackrel{\text{def}}{=} \lambda p. p(\lambda x_1 \lambda x_2 \dots \lambda x_n. x_i)$

6 Recursion

1. Question: how to present this problem by recursion through lambda calculus

What is Fact?

$\text{Fact}(n) = \text{if } (n == 0) \text{ then } 1 \text{ else } n * \text{Fact}(n-1)$

2. 不动点

x 时函数 f 的不动点, 当且仅当 $f(x) = x$

在 lambda 演算中, 任何 lambda 项都有不动点

不动点组合子是一个高阶的函数 h , 满足: 对任意 f , $(h f)$ 是 f 的一个不动点, 即 $h f = f(h f)$

3. 图灵不动点组合子 Θ :

让 $A = \lambda x. \lambda y. y(x x y)$, 则 $\Theta = A A$

Proof:

我们证对于任意 f , 有 $\Theta f = f(\Theta f)$

$\Theta f = A A f = (\lambda x. \lambda y. y(x x y)) A f$

$\rightarrow (\lambda y. y(A A y)) f$

$\rightarrow f(A A f)$

$= f(\Theta f)$

4. 丘奇不动点组合子 Y :

$Y = \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x))$